

3회차: 안드로이드 4대 컴포넌트 및 수명 주기(Lifecycle) 핵심 가이드

1. 안드로이드 4대 컴포넌트 (4 Core Components) 개요

안드로이드 애플리케이션은 독립적인 4개의 핵심 구성요소가 유기적으로 결합하여 동작합니다.

 **시험 필수 규칙:** 모든 컴포넌트는 안드로이드 시스템이 인지할 수 있도록 반드시 `AndroidManifest.xml` 파일 내부에 선언(등록)되어야만 정상 작동하며, 누락 시 앱이 즉시 강제 종료(`Crash`)됩니다.

1. **액티비티 (Activity):** 사용자 인터페이스(UI) 화면을 구성하는 가장 기본적이고 눈에 보이는 컴포넌트입니다.
2. **서비스 (Service):** UI 화면 없이 백그라운드(Background)에서 독립적으로 오랫동안 실행되는 컴포넌트입니다. (예: 음악 재생, 대용량 파일 다운로드)
3. **브로드캐스트 리시버 (Broadcast Receiver):** 시스템 전체에 전송되는 방송 신호(배터리 방전, 문자 도착 등)를 감지하고 처리하는 컴포넌트입니다.
4. **콘텐츠 프로바이더 (Content Provider):** 응용프로그램(앱) 간에 데이터를 안전하게 상호 공유하기 위한 컴포넌트입니다.

2. 액티비티(Activity)와 인텐트(Intent) 핵심 이론

2.1 인텐트(Intent)의 분류

인텐트는 컴포넌트 간에 데이터나 명령을 전달하는 메시지 객체입니다.

- **명시적 인텐트 (Explicit Intent):**
 - 호출할 대상 클래스명(또는 패키지명)을 **명확히 지정**하여 화면을 전환할 때 사용합니다. 개발자가 내부적으로 제어하는 화면 이동에 쓰입니다.
 - 예: `Intent intent = new Intent(getApplicationContext(), SecondActivity.class);`
- **암시적 인텐트 (Implicit Intent):**
 - 특정 대상을 지정하지 않고, 시스템에 원하는 액션(Action)과 **데이터(URI)** 유형을 위임하여 최적의 앱을 찾아 실행하도록 요청하는 방식입니다.
 - 예: 웹브라우저 열기(`Intent.ACTION_VIEW`), 전화걸기(`Intent.ACTION_DIAL`) 등

2.2 양방향 액티비티 데이터 전달 구조 (최신 Jetpack 표준 표준)

메인 액티비티가 서브 액티비티를 열어 파라미터를 던지고, 서브 액티비티가 연산 후 결과를 다시 메인으로 되돌려주는 아키텍처입니다. 교안의 기존 방식(`startActivityForResult`)은 현재 사용 중단(Deprecated)되었으므로, 최신 `registerForActivityResult` 방식 구조를 정확히 암기해야 합니다.

메인 액티비티 (MainActivity.java) 핵심 소스 코드

Java

```
package com.exam.component;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
import androidx.activity.result.ActivityResult;
import androidx.activity.result.ActivityResultCallback;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity{

    // [1단계] 서브 액티비티로부터 돌아올 리턴 값을 처리할 수명 주기 안전 콜백 매니저 정의
    private final ActivityResultLauncher<Intent> secondActivityLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        new ActivityResultCallback<ActivityResult>() {
            @Override
            public void onActivityResult(ActivityResult result){
                // 서브 액티비티가 정상 종료(RESULT_OK) 되었고 반환 인텐트가 존재할 때만 처리
                if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                    int sumResult = result.getData().getIntExtra("Hap", 0);
                    Toast.makeText(getApplicationContext(), "더하기 계산 결과: " + sumResult, Toast.LENGTH_LONG).show();
                }
            }
        }
    );

    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button btnCalc = findViewById(R.id.btnCalc);
        btnCalc.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v){
```

```

        // [2단계] 명시적 인텐트를 생성하고 Key-Value 형식의 부가 데이터(Extra)를 첨부
        Intent intent = new Intent(MainActivity.this, SecondActivity.class);
        intent.putExtra("Num1", 200);
        intent.putExtra("Num2", 500);

        // [3단계] 정의해 둔 런처를 통해 서브 액티비티 호출 실행
        secondActivityLauncher.launch(intent);
    }
});
}
}
/*
 * [시간 복잡도]
 * - O(1): 부가 데이터 첨부 및 액티비티 인스턴스 실행은 메모리 참조 구조이므로 상수 시간에 처리됩니
다.
 */

```

서브 액티비티 (SecondActivity.java) 결과 반환 소스 코드

Java

```

package com.exam.component;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity{

    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        // 메인 액티비티가 전달한 인텐트 객체를 수신
        Intent receivedIntent = getIntent();
        final int num1 = receivedIntent.getIntExtra("Num1", 0);
        final int num2 = receivedIntent.getIntExtra("Num2", 0);

        Button btnReturn = findViewById(R.id.btnReturn);
        btnReturn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v){
                int hap = num1 + num2;

                // 결과 데이터를 담아 보낼 빈 인텐트 생성
                Intent outIntent = new Intent();
                outIntent.putExtra("Hap", hap);

                // 메인 액티비티로 성공 결과 코드와 데이터 전송 설정
                setResult(RESULT_OK, outIntent);
            }
        });
    }
}

```

```

        finish(); // 현재 액티비티를 파괴하여 화면에서 제거하고 이전 화면으로 복귀
    }
    });
}
}

```

3. 액티비티 생명주기 (Activity Lifecycle) ★ 시험 출제 100%

액티비티는 생성되어 소멸할 때까지 안드로이드 시스템 상태에 따라 정해진 수명 주기 메서드를 차례대로 관통합니다.

수명 주기 콜백	호출 시점 및 상태 변경 특징	시험 준비용 핵심 팁
<code>onCreate()</code>	액티비티가 메모리에 최초로 생성 될 때 호출	<code>setContentView()</code> 선언 및 UI 위젯 초기화, 일회성 설정 로직을 전담하는 곳.
<code>onStart()</code>	액티비티가 화면에 보이기 직전 에 호출	화면 노출 준비 단계이며, 아직 사용자가 터치하거나 입력할 수는 없음.
<code>onResume()</code>	액티비티가 **전면 (Foreground)** 에 배치될 때 호출	사용자와 실시간으로 상호작용(Interaction) 이 가능한 유일한 상태 .
<code>onPause()</code>	다른 액티비티가 화면을 부분적으로 가릴 때 호출	포커스를 상실한 상태. 자원을 많이 소모하는 애니메이션을 멈추거나 가벼운 데이터를 임시 저장함.
<code>onStop()</code>	액티비티가 완전히 가려져 보이지 않을 때 호출	백그라운드로 완전히 밀려난 상태. 자원 누수를 막기 위해 무거운 리소스를 해제해야 함.
<code>onRestart()</code>	<code>onStop()</code> 상태의 액티비티가 소멸되지 않고 다시 호출될 때	<code>onStop</code> → <code>onRestart</code> → <code>onStart</code> 구조로 이어짐.
<code>onDestroy()</code>	액티비티가 메모리에서 소멸 될 때 최종 호출	사용자가 <code>finish()</code> 를 호출했거나 자원 부족으로 OS가 액티비티를 영구 파괴하는 단계.

4. 백그라운드 컴포넌트: 서비스 & 브로드캐스트 리시버

4.1 서비스 (Service) 생명주기

- UI 화면 없이 백그라운드 처리를 전담하는 컴포넌트입니다.
- **핵심 실행 흐름:** `onCreate()` → `onStartCommand()` → `onDestroy()`
- **⚠ 오답 방지:** 서비스가 실질적인 백그라운드 연산 업무를 시작하고 수행하는 핵심 메서드는 `onCreate`가 아니라 `onStartCommand(Intent, int, int)`입니다. 업무가 완료되면 메

모리 관리를 위해 반드시 내부에서 `stopSelf()` 를 호출해 주어야 안전합니다.

4.2 브로드캐스트 리시버 (Broadcast Receiver)

- 안드로이드 운영체제(OS)가 사방으로 뿌리는 방송 신호를 캐치하여 반응하는 컴포넌트입니다. (예: 배터리 상태 변화 `ACTION_BATTERY_CHANGED`)
- 신호가 도달하면 오직 `onReceive(Context, Intent)` 메서드가 깨어나 로직을 실행합니다.
- **동적 등록 보안 수칙:** 자원 낭비를 방지하기 위해 액티비티의 `onResume()` 에서 `registerReceiver()` 로 리시버를 등록하고, 화면을 나가는 `onPause()` 시점에 `unregisterReceiver()` 로 반드시 해제해 주는 패턴이 철칙입니다.

4.3 콘텐츠 프로바이더 (Content Provider)

- 앱 간에 독립된 데이터베이스나 파일 정보를 상호 공유할 수 있도록 해주는 보안 허브 인터페이스입니다.
- 데이터 식별 및 경로 추적을 위해 주소 역할을 하는 **URI(Uniform Resource Identifier)** 스키마(`content://...`)를 필수적으로 적용합니다.
- 타 앱의 프로바이더 데이터를 제어할 때는 직접 접근하지 못하고, 중계 객체인 `ContentResolver` 클래스의 기본 CRUD 메서드인 `query()` , `insert()` , `update()` , `delete()` 를 호출하여 간접적으로 제어합니다.